Práctica 4 – Programa para visualizar por display cifras decimales

Objetivo

Escribir un programa en lenguaje ensamblador para visualizar ininterrumpidamente a través de un display de 7 segmentos de cátodo común las 10 cifras decimales en orden creciente comenzando por el 0. Las cifras irán cambiando cada 0.5 segundos. Al llegar al 9, se volverá al 0.

Contenido del proyecto esqueleto (starter_kit)

Para comenzar a escribir el código, se os proporciona un proyecto que contiene el esquemático **esquematico.DSN** que aparece en la Figura 1. Incluye, además del microcontrolador LPC2103, un hiperterminal conectado al periférico UART0 y un display de 7 segmentos de cátodo común (**7SEG-MPX1-CC**). Los ánodos de los 7 leds que componen el display están conectados a los pines 7-14 según se indica en la figura[1].

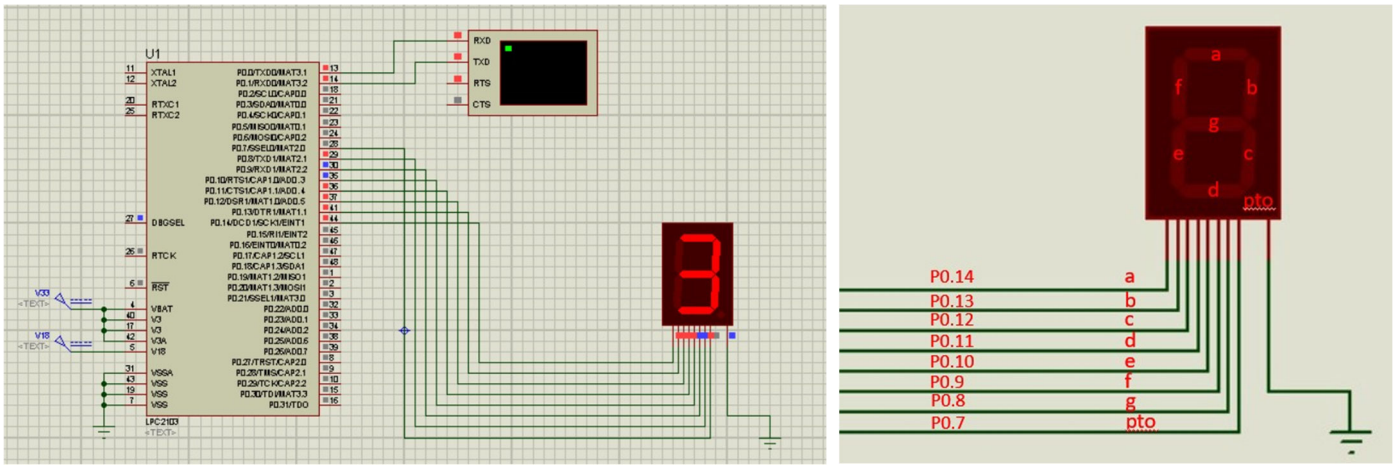

Figura 1 – Esquemático del proyecto. Ampliación de las conexiones del display

Por su parte, el fichero **crt.S** contiene la tabla de vectores, rutina de reset (**Reset_Code**), algunas rutinas de visualización y entrada de datos y la rutina **_delay_loop** que utilizaremos para el diseño de la espera de 0.5 s. En la rutina de reset se establece la frecuencia de la señal de reloj de la CPU en **58.9824 MHz**, ya que se configura el PLL para que proporcione una señal de reloj con una frecuencia 4 veces mayor que la del cristal de cuarzo (14.7456 MHz). También, en las líneas 145-147 se carga PINSEL0 con el número 0x5, manteniéndose PINSEL1 en su valor por defecto (0x0). Con esta acción, todos los pines de propósito general se encuentran conectados al **legacy GPIO** salvo los pines 0 y 1 que están conectados a la UART0.

Por último, el fichero **main.S** contiene el programa de bienvenida de las prácticas anteriores. Antes de comenzar a modificar el código, compilaremos y realizaremos la simulación de este proyecto y observaremos que el mensaje "\r\nBienvenidos al LPC2103\r\n-- Esqueleto de programa en ensamblador --\r\n" aparece en el hiperterminal. Pausando el programa, desplegaremos la ventana **Watch Window** desde donde podemos visualizar los valores de los puertos: **PINSEL0**, **PINSEL1**, **IOxPIN,** etc…

Desarrollo del programa

Para realizar la tarea indicada en el objetivo se debe primeramente definir los valores que tienen que tomar los segmentos del display para visualizar las cifras decimales. Para ello, en el fichero **main.S**, tras el string de bienvenida definiremos 10 bytes con la directiva **.byte**. Cada byte contendrá el valor de los 7 segmentos del display para cada cifra decimal[2]. Así, por ejemplo, para visualizar el 0, los segmentos comprendidos entre a y f deben estar en ON y el segmento g en OFF. Teniendo en cuenta a qué pin concreto se encuentra conectado cada segmento, vemos que con el valor **0xFC** que equivale al binario 11111100 podemos visualizar el 0. Procediendo de igual manera con el resto de

[1] Para saber a qué segmento corresponde cada pin del display, basta con situar el ratón sobre dicho pin. El texto que aparece en la parte inferior de la ventana nos indicará su nombre.

[2] En el fichero main.S aparece como ejemplo la cifra 0.

las cifras decimales obtendremos una matriz compuesta por 10 bytes (uno por cada cifra decimal) etiquetada con **seg7**.

Tras diseñar la matriz de valores a visualizar, proceder según los pasos:

1. Configurar el Legacy GPIO para que los pines adecuados sean salidas. Por defecto, inicialmente todos son entradas.
2. Utilizar un puntero que vaya recorriendo la matriz y enviando cada cifra al display. Entre cifra y cifra se debe llamar a la subrutina **_delay_loop** para implementar una espera de 0.5 segundos. Al llegar al final de la matriz, el puntero debe volver al primer byte (el del 0).
3. Para calcular el valor del parámetro que hay que pasar a la subrutina **_delay_loop** en r0, analizar su código teniendo en cuenta la frecuencia a la que funciona el ARM7 y el número de ciclos de reloj que tardará su ejecución.[3]

[3] Nota Importante: en Proteus existe un bug que afecta al tiempo de simulación que se puede visualizar en el texto situado en la parte inferior de la ventana. Cuando el cálculo del parámetro r0 de la subrutina _delay_loop sea correcto, el tiempo que el simulador nos dirá que transcurre será de 400 ms en vez de 500 ms. Sin embargo, se puede estimar el tiempo transcurrido correctamente a partir del número de ciclos de reloj entre visualización y visualización.